

Méthode de Galerkin Discontinue sur des Éléments Pyramidaux

Stage de 2^{ème} année – ENSE3 / INRIA Bordeaux

Julien Royer

INRIA Bordeaux Sud-Ouest

Juin – Septembre 2025

Maître de stage : Aurélien Citrain



Plan de la présentation

- 1 Contexte du stage
- 2 État de l'art
- 3 Bases fonctionnelles étudiées
- 4 Implémentation numérique
- 5 Intégration dans GEOS
- 6 Conclusion

Contexte du stage

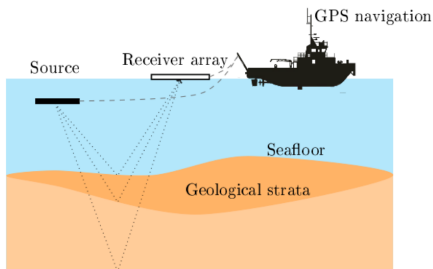
Contexte général

Cadre :

- Stage INRIA Bordeaux, équipe Makutu
- Collaboration avec TotalEnergies (HPC, imagerie sismique)
- Travail intégré au code open-source GEOS



L'équipe Makutu



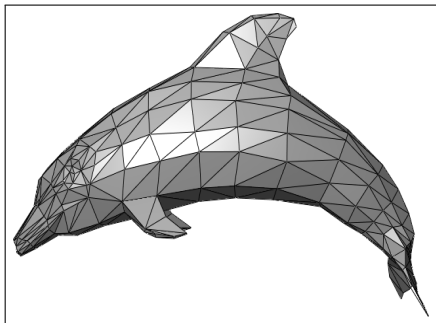
- Spécialisée en modélisation et simulation d'ondes.
- Applications : géophysique, océanographie, stockage géologique du CO_2 .
- Forte expertise en méthodes numériques : FEM, SEM, DG.

Méthode des Éléments Finis (FEM)

But : *Calculer les valeurs d'un champ régi par une équation différentielle (EDO, EDP)*

Principe

- Le domaine est découpé en éléments simples :
 - triangles / quadrilatères en 2D,
 - tétraèdres / hexaèdres en 3D.
- La solution est approchée par des **fonctions de base continues**.
- On impose la continuité entre les éléments.



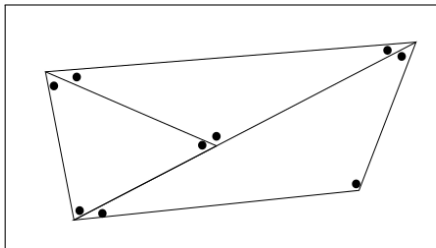
Méthode de Galerkin Discontinue (DG)

Principe

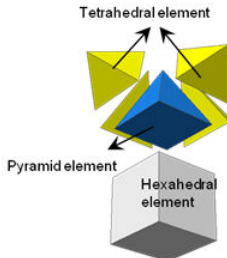
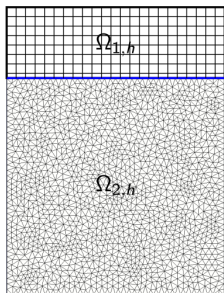
- Les fonctions de base sont **discontinues** entre éléments.
- Les éléments communiquent via des **flux numériques**.

Avantages

- Très performante à ordre élevé.
- Naturellement **parallélisable**.



Objectifs techniques du stage



- Développer les bases fonctionnelles pour l'élément pyramidal.
- Intégrer la méthode DG pyramidale dans GEOS.
- En vue d'implémenter les maillages hybrides

État de l'art

Équation d'onde considérée

On considère l'équation d'onde au second ordre suivante :

$$\left\{ \begin{array}{l} \frac{1}{c^2} \partial_{tt} p(x, t) - \Delta p(x, t) = f(x, t), \quad \text{dans } \Omega \times (0, T], \\ \alpha \partial_t p(x, t) + \frac{\partial p}{\partial n}(x, t) = 0, \quad \text{sur } \partial\Omega \times (0, T], \\ p(x, 0) = p_0(x), \\ \partial_t p(x, 0) = p_1(x), \end{array} \right. \quad (1)$$

où :

- $p(x, t) \in \mathbb{R}$ pression,
- c vitesse de propagation de l'onde,
- $f(x, t)$ source,
- α paramètre de condition aux limites : $\alpha = 0$ condition de Neumann, $\alpha = \frac{1}{c}$ absorbante du premier ordre,
- $\Omega \subset \mathbb{R}^d$ domaine spatial.

Principe de la méthode DG

- Approximation polynomiale **discontinue** par élément
- Communication via **flux numériques**

$$p_h \in V_h := \prod_K \mathbb{P}^N(K),$$

$\mathbb{P}^N(K)$ espace des polynômes de degré $\leq N$ sur l'élément K .

$$p_h(x, t)|_K = \sum_{j=1}^{N_p} p_j^K(t) \phi_j(x),$$

- $\{\phi_j(x)\}_{j=1}^{N_p}$ base polynomiale locale sur K ,
- N_p est le nombre de degrés de liberté,
- $p_j^K(t)$ coefficients temporels.

Formulation DG : sous forme matricielle

Sur chaque élément K , le système peut s'écrire sous forme semi-discrète :

$$\mathbf{M} \partial_{tt} \mathbf{p}_h^K = -\mathbf{S} \mathbf{p}_h^K + \mathbf{F}^{\partial K} \mathbf{p}_h^K + \mathbf{b}^K,$$

- $\mathbf{M}$: matrice de masse,
- $\mathbf{S}$: matrice de raideur,
- $\mathbf{F}^{\partial K}$: contributions de flux numériques sur les bords de K ,
- $\mathbf{b}^K$: contributions de la source volumique.

Matrices locales en DG et rôle des fonctions de base

Matrice de masse

$$M_{ij} = \int_K \phi_i \phi_j dK$$

Matrice de raideur

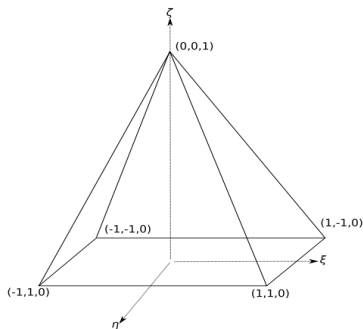
$$S_{ij} = \int_K \nabla \phi_i \cdot \nabla \phi_j dK$$

Flux numériques (interfaces)

$$F_{ij}^{\partial K} = \sum_{f \subset \partial K} \int_f \widehat{\nabla \phi_j \cdot n} \phi_i df$$

$\Rightarrow$ **Le choix des fonctions ϕ_j est crucial !**

Application aux éléments pyramidaux



- Géométrie non tensorielle
- Singularité au sommet
- Bases fonctionnelles rationnelles

Bases fonctionnelles étudiées

Méthode 1 : Base nodale Bergot-Cohen-Duruflé (2010)

Basée sur les polynômes de Jacobi :

- Construction d'une base modale **orthogonale** :

$$\psi^{i,j,k}(x,y,z) = P_i^{(0,0)}\left(\frac{x}{1-z}\right) P_j^{(0,0)}\left(\frac{y}{1-z}\right) (1-z)^{\max(i,j)} P_k^{(2\max(i,j)+2,0)}(2z-1)$$

pour les triplets (**modes** avec r m'ordre du maillage) :

$$0 \leq i \leq r, \quad 0 \leq j \leq r, \quad 0 \leq k \leq r - \max(i,j)$$

- Ce point de départ permet de profiter des propriétés intéressantes de l'**orthogonalité**.

Méthode 1 : Base nodale Bergot-Cohen-Duruflé (2010)

Passage à une base nodale :

- Par la matrice de Vandermonde (VDM) :

$$\text{VDM}_{i,j} = \psi_j(M_i)$$

$\implies$ la matrice des fonctions de base **nodales** N :

$$N = \text{VDM}^{-1} \cdot \psi$$

- Les lignes de N sont nos fonctions ϕ_i
- Condition d'interpolation nodale : $\phi_i(M_j) = \delta_{ij}$.
- Matrice de masse diagonalisable.

Méthode 2 : Base modale Bernstein-Bézier

Basée sur les polynômes de Bernstein-Beziers (BB) :

$$B_i^n(t) = \binom{n}{i} t^i (1-t)^{n-i}, \quad \text{pour } i = 0, 1, \dots, n \text{ avec } n \text{ l'ordre des polynômes.}$$

Polynômes de BB en 3D (héxaèdres) :

$$B_{ijk}(a, b, c) = B_i^{n-k}(a) B_j^{n-k}(b) B_k^n(c)$$

pour

$$0 \leq k \leq n, \quad 0 \leq i, j \leq n - k$$

Méthode 2 : Base modale Bernstein-Bézier

Changement de coordonnées pour "aplatir" le cube :

$$\begin{cases} r = x(1 - z) \\ s = y(1 - z) \\ t = z \end{cases}$$

On obtient ainsi directement l'espace de fonctions de base pour un élément pyramidal :

$$\phi_{ijk}(r, s, t) = B_{ijk}\left(\frac{r}{1-t}, \frac{s}{1-t}, t\right)$$

- **Base non nodale !** ($\phi_i(M_j) \neq \delta_{ij}$)

Méthode 3 : Bernstein + algorithme de De Casteljau

Evaluation des polynômes de BB avec l'**algorithme de De Casteljau** :

Initialisation : $b_i^{(0)} := b_i$, pour $i = 0, \dots, n$

Récurrence : $b_i^{(r)} := (1 - t) b_i^{(r-1)} + t b_{i+1}^{(r-1)}$, pour $i = 0, \dots, n - r$,
 $r = 1, \dots, n$

Résultat : $B(t) = b_0^{(n)}$

$\implies$ algorithme **numériquement plus stable** que l'évaluation directe de la formule de Bernstein pour un calcul aux ordres élevés

Implémentation numérique

Choix du langage et structure

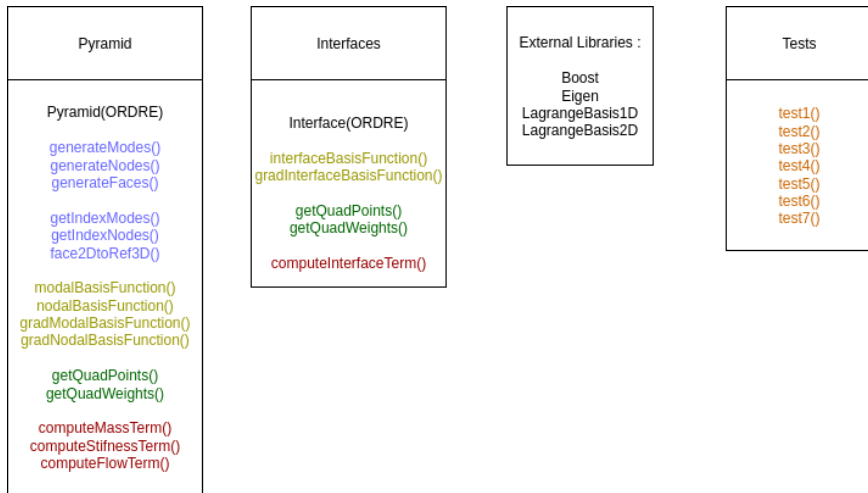
Choix du langage

- Implémentation en **C++** (intégration future dans GEOS).
- Langage compilé : **rapidité**, contrôle mémoire.
- Support natif du **parallélisme**.

Structure des codes

- **Définition des géométries** : élément pyramidal, nœuds, modes, coordonnées locales.
- **Bases d'approximation** : Jacobi, Bernstein, avec/sans De Casteljau.
- **Intégration numérique** : quadratures Gauss–Legendre, Gauss–Lobatto, Hesthaven.
- **Assemblage** : matrices masse/rigidité/flux.
- **Solveur** : cas tests : onde, diffusion.

Diagramme UML simplifié



Tests de validation

Objectif : valider matrices et bases avant comparaison.

- **Masse (symétrie)** : $M = M^T$
- **Partition unité** : $\sum_i \varphi_i = 1$
- **Volume** : $\mathbf{1}^T M \mathbf{1} = |K| = 4/3$
- **Nodalité (BCD)** : $\varphi_i(x_j) = \delta_{ij}$
- **Raideur** : $\mathbf{1}^T S \mathbf{1} = 0$
- **Flux masse** : $\mathbf{1}^T M^\Gamma \mathbf{1} = 4$
- **Flux raideur** : $\mathbf{1}^T B^\Gamma \mathbf{1} = 0$

Comparaison des méthodes

Méthode	Base	Coût	Qualité
Bernstein	modale	+++	+
Bernstein + DeC	modale	+	++
BCD	nodale	++	+++

Légende : + = faible ; ++ = moyen ; +++ = élevé.

Choix final

Méthode retenue

Bergot – Cohen – Duruflé

- Compatible avec les quadratures Gauss–Lobatto
- Cohérence avec les hexaèdres de GEOS
- Matrices mieux conditionnées

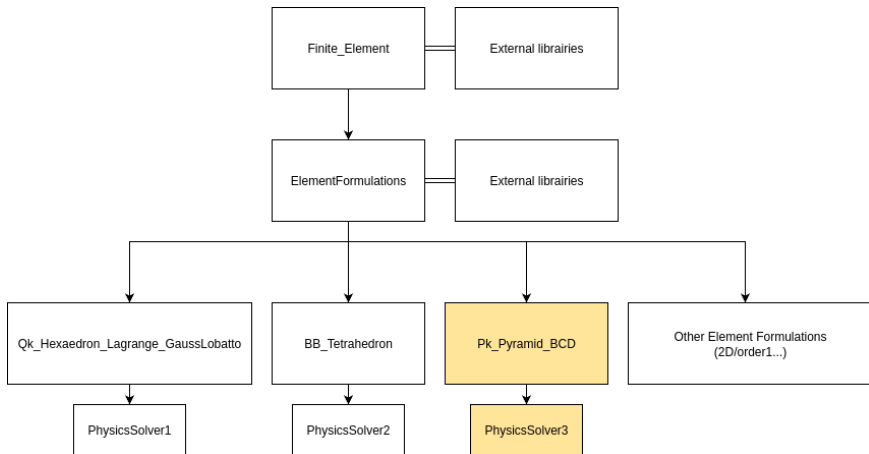
Intégration dans GEOS

Intégration dans le code GEOS

- Factorisation et optimisation des calculs
- Adaptation à la philosophie GEOS : calcul à la volée, calculs sur GPU
- Ouverture d'une **pull request** officielle :

<https://github.com/GEOS-DEV/GEOS/pull/3793>

Structure de la branche



Conclusion

Bilan

- Travail en autonomie
- Méthode de travaux recherche
- Contribution réelle à un projet de collaboration internationale porté par plusieurs acteurs
- Compétences en C++, HPC, Git, Linux, Docker...
- Résultats concrets implémentés

Merci pour votre attention